Main Memory

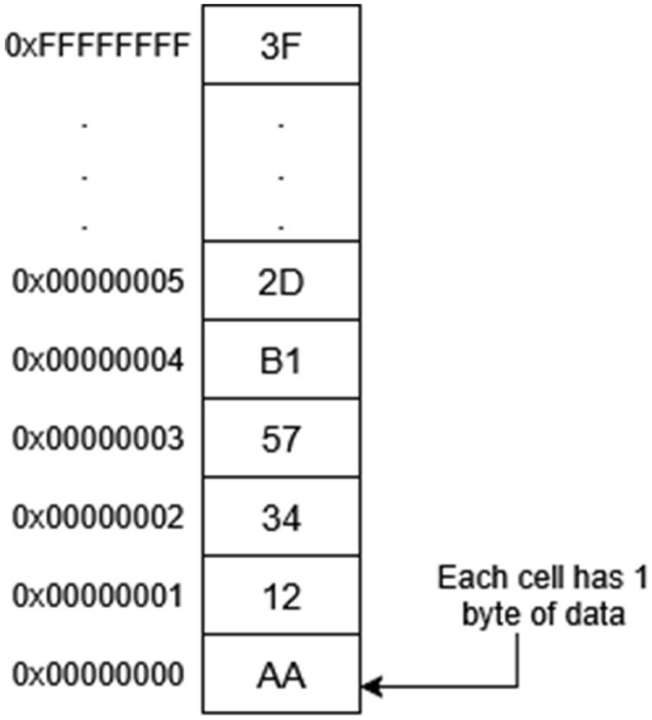
Introduction

- The CPU can only execute instructions and access data that is in **main memory (RAM)**
  - The challenge is efficiently and securely managing this limited resource among multiple processes
- **Goals of Memory Management:**
  - **Relocation:** Allow processes to be loaded into any part of memory
  - **Protection:** Prevent one process from accessing another process's memory
  - **Sharing:** Allow controlled sharing of memory between processes
  - **Logical Organization:** Provide an abstract, logical view of memory for the programmer
- Focus of this section covers how addresses are bound, the basic swapping mechanism, contiguous memory allocation, and the transition to non-contiguous methods like paging and segmentation

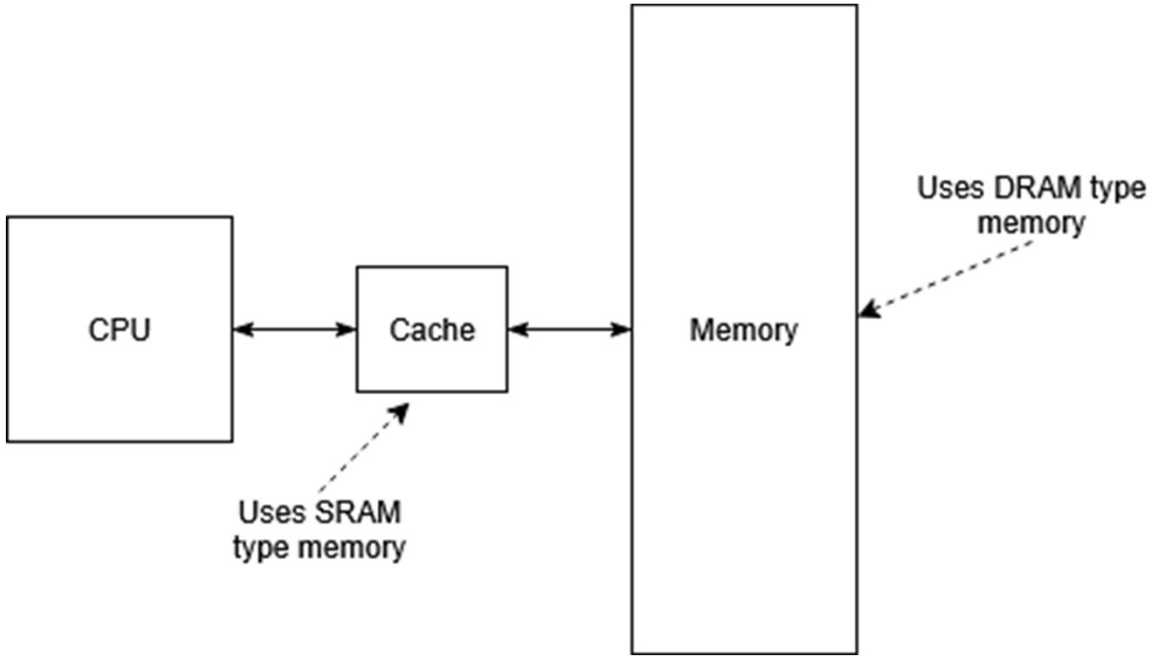
Basic Concepts and Memory Hardware

Basic Concepts in Memory

- Memory consists of a large array of bytes, each with its own address
- CPU fetches instructions from memory according to the value of the program counter
- A typical instruction-execution cycle:
  - CPU fetches an instruction from memory
  - Second, instruction is then decoded and may cause operands to be fetched from memory
  - After the instruction has been executed on the operands, results may be stored back in memory
- Completing a memory access may take many cycles of the CPU clock
  - The processor normally needs to **stall**, since it does not have the data required to complete the instruction that it is executing
  - The remedy is to add fast memory between the CPU and main memory, typically on the CPU chip for fast access which is called a **cache**
    - Speeding up memory access without any operating system control

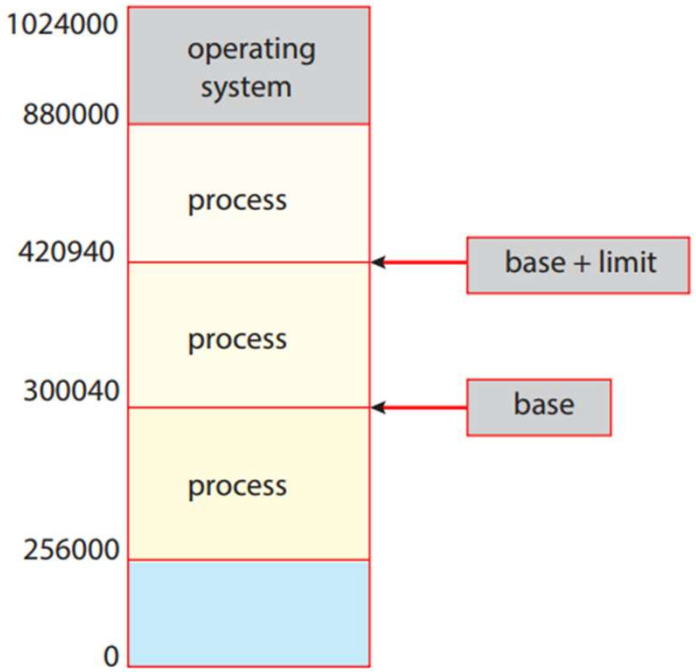


Cache Memory



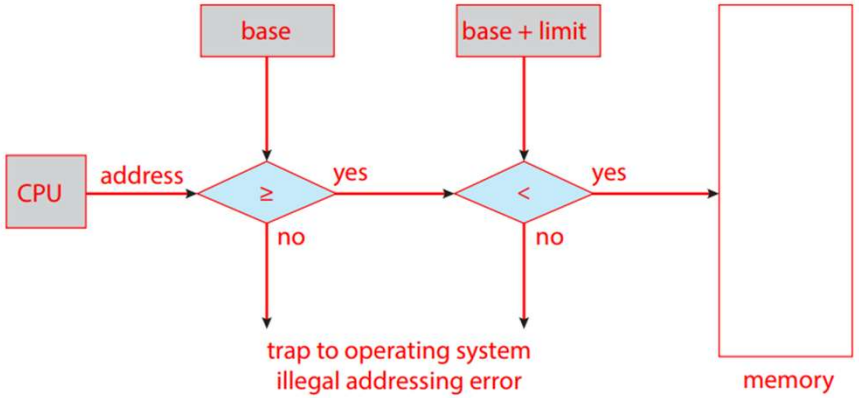
Basic Hardware

- Memory protection is achieved using **Base and Limit Registers**
  - The CPU generates a logical address, and the hardware checks if it is within the valid range defined by these registers
- **Base Register:** Holds the smallest valid physical address
- **Limit Register:** Specifies the size of the valid range



Basic Hardware

- **Protection Mechanism**
  - **Enforced by CPU Hardware:** The **CPU hardware** checks every address generated in **user mode** against specific registers (**base and limit**)
  - **Unauthorized Access:** Any attempt to access memory outside a program's allocated space (OS or other users' memory) results in a **trap** to the Operating System (OS)
  - **Result:** The OS treats this trap as a **fatal error**, preventing accidental or deliberate modification of its code/data or that of other users



Basic Hardware

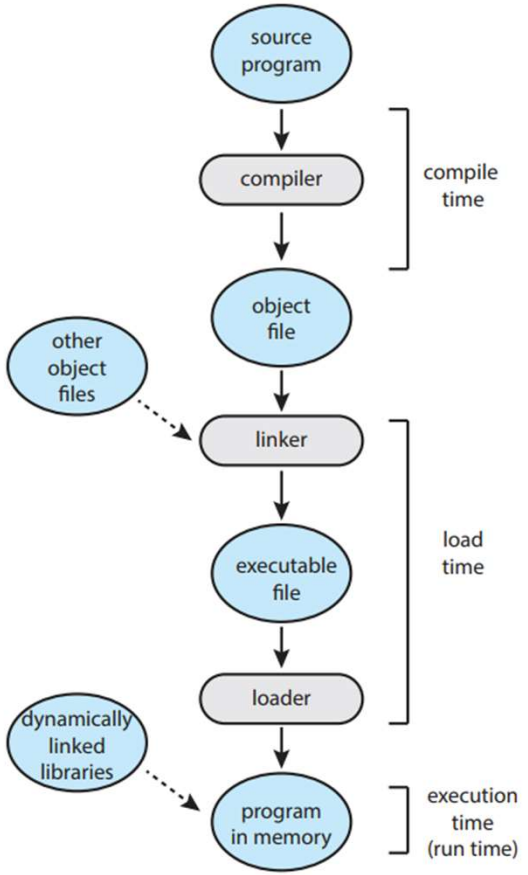
- **Register Control (Security)**
  - **Privileged Instruction:** The **base and limit registers** can only be loaded or changed by the OS using a **special privileged instruction**
  - **Kernel Mode Only:** Since privileged instructions run only in **kernel mode**, and only the OS executes in kernel mode, only the OS can set memory boundaries
  - **Goal:** This prevents user programs from altering the registers to gain unauthorized access to memory

Basic Hardware

- **OS Unrestricted Access**
  - **Kernel Mode Privilege:** When executing in **kernel mode**, the OS has **unrestricted access** to both operating-system memory and all users' memory
  - This access is necessary for the OS to perform vital services, including:
    - Loading user programs
    - Dumping programs in case of errors
    - Handling system calls
    - Performing I/O operations

Address Binding

- The process of mapping program addresses to physical memory addresses
- This can happen at three stages:
  - **Compile Time:** Addresses are fixed if the memory location is known (absolute code)
  - **Load Time:** Addresses are bound when the program is loaded into memory (relocatable code)
  - **Execution Time:** Addresses are bound during execution by the hardware (most common), allowing a process to be moved during its lifetime



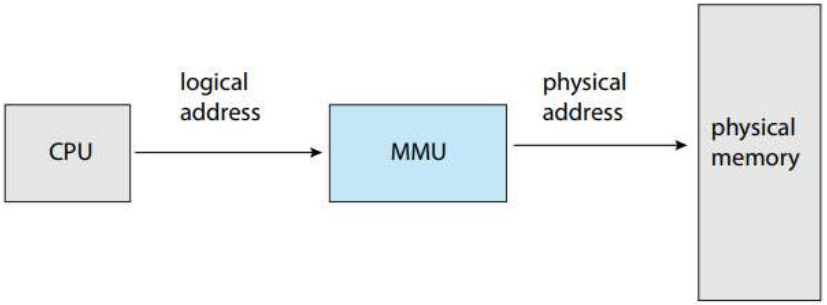
Logical vs. Physical Address

- **Logical (Virtual) Address:** The address generated by the CPU
  - This is the **virtual address** that a program sees and uses
  - It is a conceptual address space that is independent of the actual hardware configuration

- **Physical Address:** The actual address seen by the memory hardware
  - This address is used by the memory controller to locate the actual memory cell in RAM
  - User program never accesses the real physical addresses

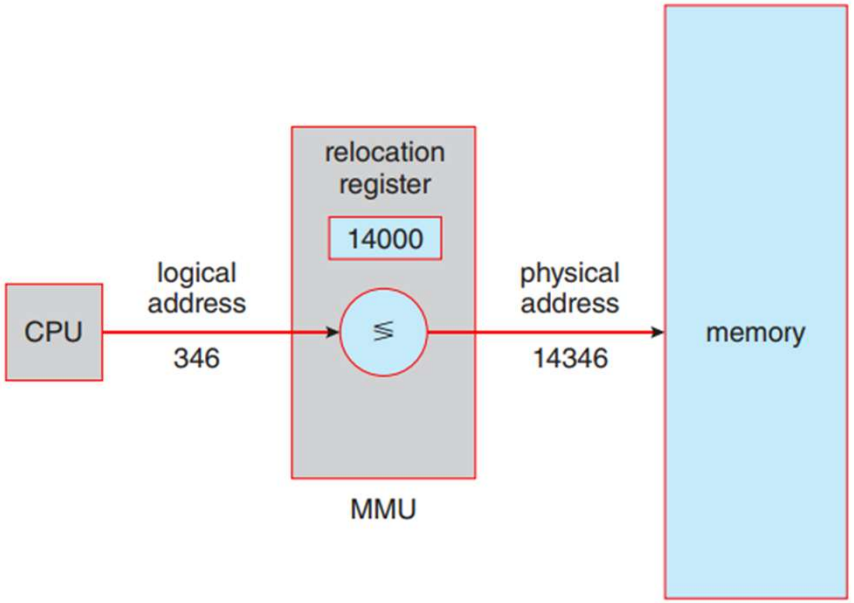
- **Decoupling:** The key distinction is that the logical address space is **decoupled** from the physical address space, allowing the OS to manage memory more flexibly and securely

- **Memory-Management Unit (MMU):** A hardware device that handles the run-time mapping from logical to physical addresses
  - This is critical for address binding at execution time



Memory Management Unit

- The **Memory-Management Unit (MMU)** is a dedicated hardware device that performs the crucial run-time mapping from **logical (virtual) addresses** to **physical addresses**
- The MMU is the component that makes **execution-time address binding** possible.
  - It ensures that the memory references generated by a program are translated to valid physical addresses before being sent to the memory bus
- The MMU uses the **Base Register** (Relocation Register) to transform the logical address into a physical address (Physical Address = Logical Address + Base Register Value)
  - It also enforces protection using the **Limit Register**
- **MMU in Paging/Segmentation:** In more complex schemes, the MMU utilizes the **Page Table** or **Segment Table** to translate addresses
- **MMU and TLB:** The MMU also often contains the **Translation Look-aside Buffer (TLB)**, a high-speed cache used to accelerate the translation process



Dynamic Loading

- Dynamic Loading is a memory optimization technique where a routine (function, module) is **not loaded** into main memory until it is actually called by the program during execution
  - All routines are kept on disk in a relocatable load format
  - The main program is loaded and executed
  - When a routine needs to call another, the loader checks if the routine is already in memory
  - If not, the relocatable linking loader loads the routine and updates the program's address tables.
- Increases effective memory utilization
  - Unused routines (like complex error handling routines or rarely used features) are never loaded, saving valuable RAM space
  - This is particularly useful for handling large amounts of infrequently used code, such as error routines

Dynamic Linking

- **Dynamically linked libraries** (**DLLs**) are system libraries that are linked to user programs when the programs are run
- **Dynamic Linking** is similar to Dynamic Loading where linking (the process of combining code) is postponed until **execution time**.
  - The compiler includes only a small piece of code, called a **stub**, in the final executable file for each dynamically linked library routine
  - When the stub is executed, it first checks if the library routine is loaded
  - If not, it loads the dynamic library (DLL in Windows, shared object in Linux)
- **Benefit:** Allows **shared libraries**
  - Multiple processes can share a single copy of the library code in memory, saving disk space and physical memory
  - This is the standard method for linking system libraries

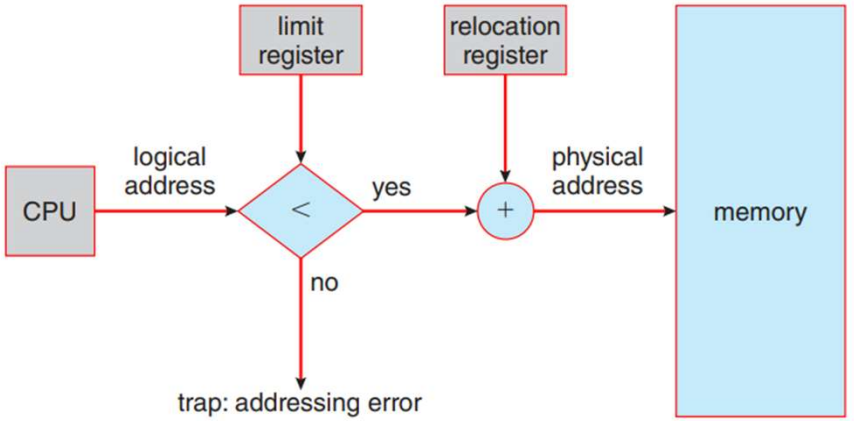
Contiguous Memory Allocation

Contiguous Memory Allocation

- Efficiently manage main memory to accommodate both the Operating System (OS) and multiple user processes.
- **Basic Partitioning:** Memory is typically divided into two main sections:
  - The Operating System
  - User Processes
- **OS Placement:**
  - Can reside in low or high memory addresses.
  - Many modern systems (e.g., Linux, Windows) place the OS in **high memory**.
- **Contiguous Memory Allocation:**
  - An early method for allocating memory.
- **Definition:** Each process is contained within a *single, continuous section* of memory.
- Before discussing allocation strategies, **memory protection** must be addressed.

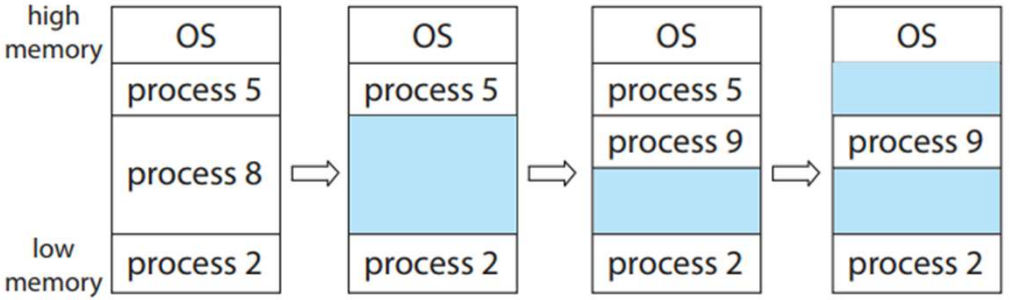
Memory Protection

- Prevent a process from accessing memory it does not own.
- **Hardware Solution:** Use two special-purpose registers:
  - **Relocation Register:** Stores the *smallest physical address* (the "base")
    - *Example: 100040*
  - **Limit Register:** Stores the *range (size)* of the logical addresses
    - *Example: 74600*
- **Operating System Role (Context Switch):**
  - When the CPU scheduler selects a process, the **dispatcher** loads the correct relocation and limit values for that specific process.
  - This ensures every address generated is checked against the *current process's* boundaries
- **Benefits:**
  - Protects the OS from user processes
  - Protects user processes from each other
- **Flexibility:** Allows the OS size to change dynamically (e.g., loading/unloading device drivers)



Memory Allocation

- **The Dynamic Storage-Allocation Problem**
  - **Problem:** How to satisfy a request of size *n* from a list of free memory "holes"?
  - **Context:** This is a core part of contiguous memory allocation, where the OS maintains a list of available (free) blocks of memory
- **Common Strategies:**
  - **First-fit:**
    - Allocate the *first* hole that is big enough.
    - Searching stops as soon as a suitable hole is found.
    - It is generally the fastest allocation method.
  - **Best-fit:**
    - Allocate the *smallest* hole that is big enough.
    - Must search the entire list of holes (unless sorted by size).
    - *Result:* Produces the smallest leftover hole.
  - **Worst-fit:**
    - Allocate the *largest* hole.
    - Must also search the entire list.
    - *Result:* Produces the largest leftover hole, which might be more useful (for future, larger requests) than the small leftover hole from Best-fit.



- Simulations show that both **First-fit** and **Best-fit** are generally better than Worst-fit in terms of decreasing both time and storage utilization.
- First-fit and Best-fit are generally preferred, but there is no clear winner between them for all cases.

Fragmentation

- **Problem:** Contiguous memory allocation schemes (First-Fit, Best-Fit) lead to fragmentation, which is wasted memory

- **Two Types of Fragmentation:**
- **External Fragmentation:**
  - **Definition:** Occurs when there is enough *total* memory space to satisfy a request, but the available space is broken into many small, non-contiguous pieces ("holes")
  - **Result:** A new process may not be loadable, even if there's enough free memory in total, because no single hole is large enough
  - **Severity:** Statistical analysis suggests that, given *N* allocated blocks, another 0.5*N* blocks may be lost to fragmentation (the "50-percent rule")
- **Internal Fragmentation:**
  - **Definition:** Occurs when allocated memory is larger than the requested memory; the unused space *within* an allocated partition is internal fragmentation
  - For example, if a process requests 18,462 bytes but is allocated a fixed partition of 18,464 bytes, 2 bytes are wasted internally
  - General approach to avoiding this problem is to break the physical memory into fixed-sized blocks and allocate memory in units based on block size

Solution to Fragmentation

- **Compaction:**
  - **Goal:** Shuffle memory contents to place all free memory together into one large, contiguous block
  - **Requirement:** Only possible if relocation is dynamic (addresses bound at execution time)
  - **Drawback:** Can be a very time-consuming operation
- **Paging:** (The modern solution)
  - **Goal:** Allow the physical address space of a process to be **noncontiguous**
  - **Result:** A process can be allocated physical memory wherever frames are available, eliminating the need for a single large hole

Paging

Paging: A Solution to External Fragmentation

- **Goal:** A memory-management scheme that permits a process's physical address space to be **noncontiguous**.
- **Key Idea:** Paging avoids the problem of fitting variably sized memory chunks onto the backing store.
- **Benefit:** Eliminates external fragmentation and the need for compaction. Paging is the most common memory-management technique.

Fundamental Concepts of Paging

- **Frames:** Physical memory is broken into fixed-sized blocks
- **Pages:** Logical memory is broken into blocks of the same size
- **Operation:** When a process is to be executed, its pages are loaded into *any* available memory frames
  - The backing store (disk) is also divided into blocks of the same size
- **Address Translation:**
- Every address from the CPU is divided into two parts:
  - **Page number (p):** Used as an index into a **page table**
  - **Page offset (d):** The location within the page



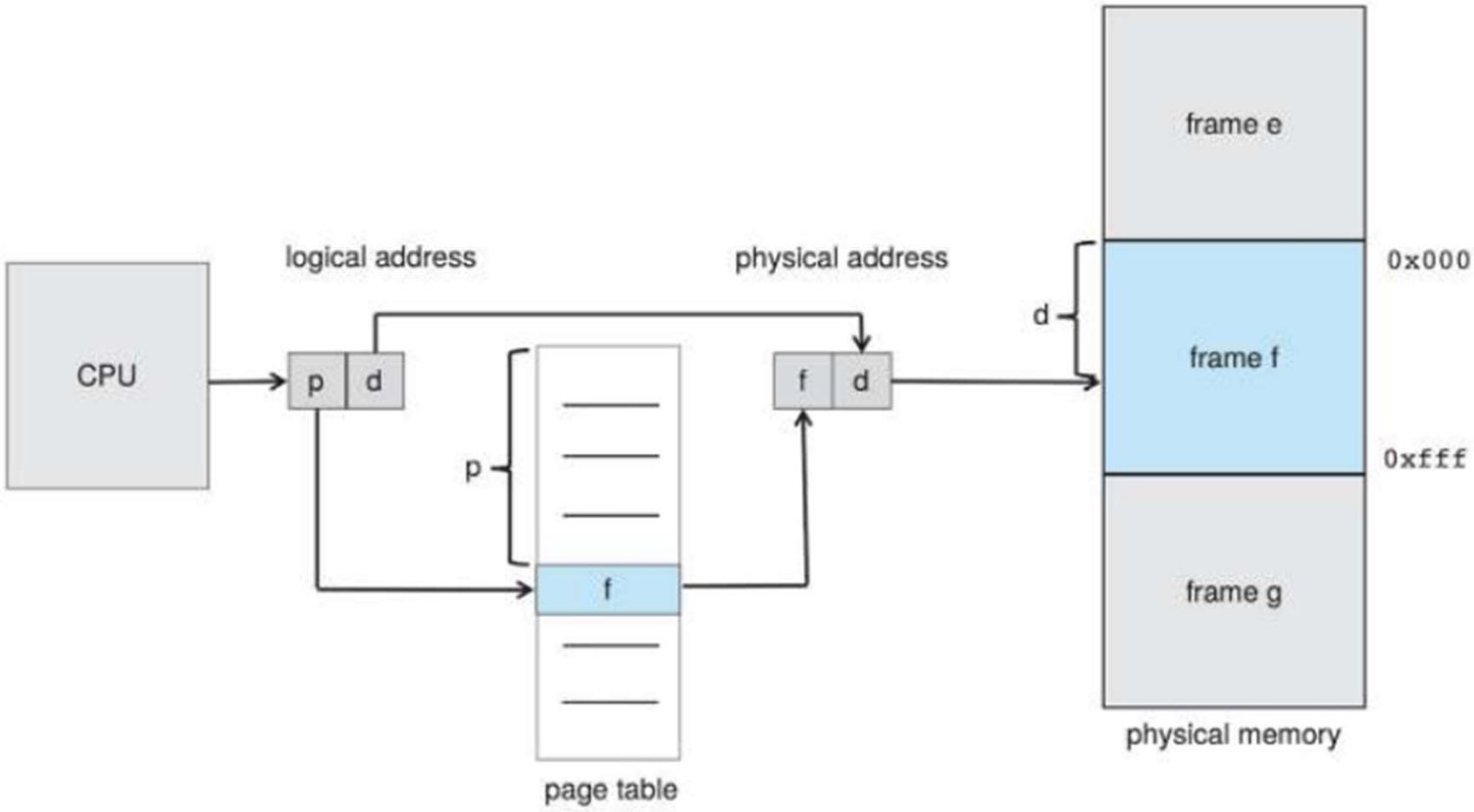
- The **page table** contains the base address of each frame in physical memory

Mapping Logical to Physical Addresses

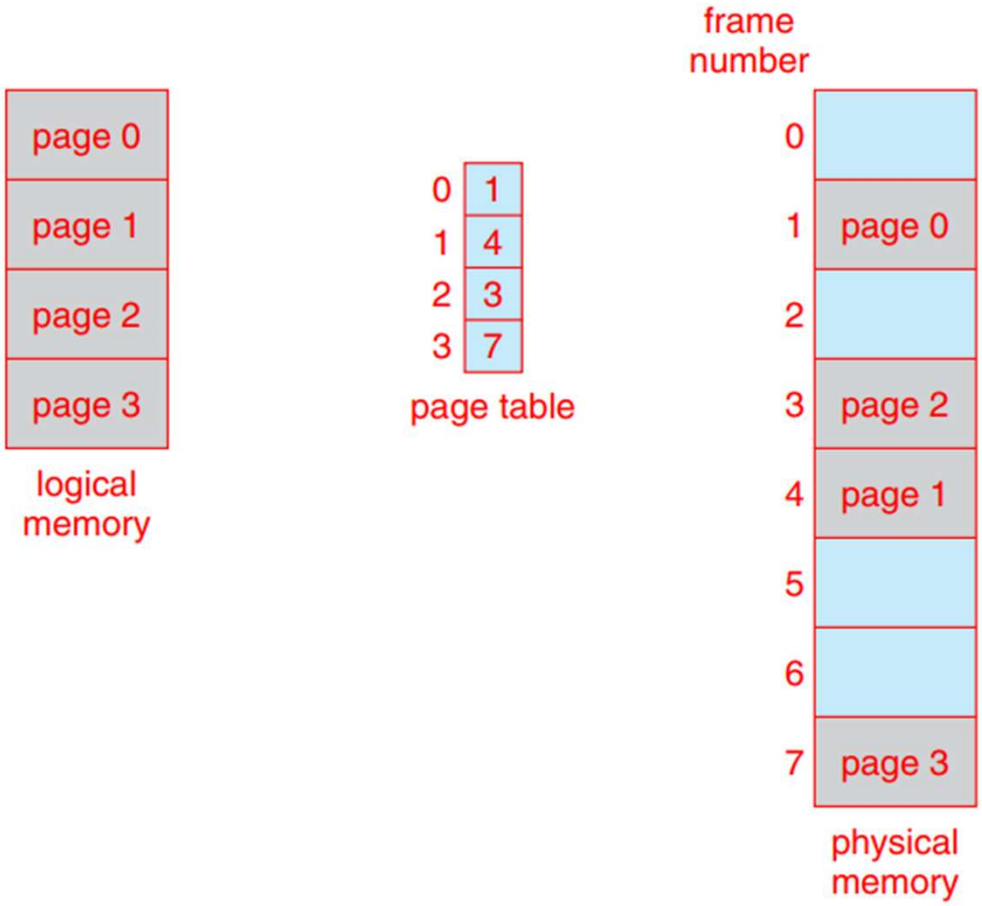
1. The CPU generates a logical address (page number p, offset d)
2. The Memory Management Unit (MMU) uses p as an index to find the corresponding entry in the process's page table
3. The page table entry provides the physical **frame number (f)**
4. The frame number f is combined with the page offset d to form the final physical memory address

- **Key Feature**: The page offset is never changed. The mapping only changes the page number to a frame number
- **Internal Fragmentation**: Paging can have internal fragmentation
  - If a process doesn't use a full page, the unused portion of the frame is wasted

Paging Hardware



Paging Model of Logical and Physical Memory



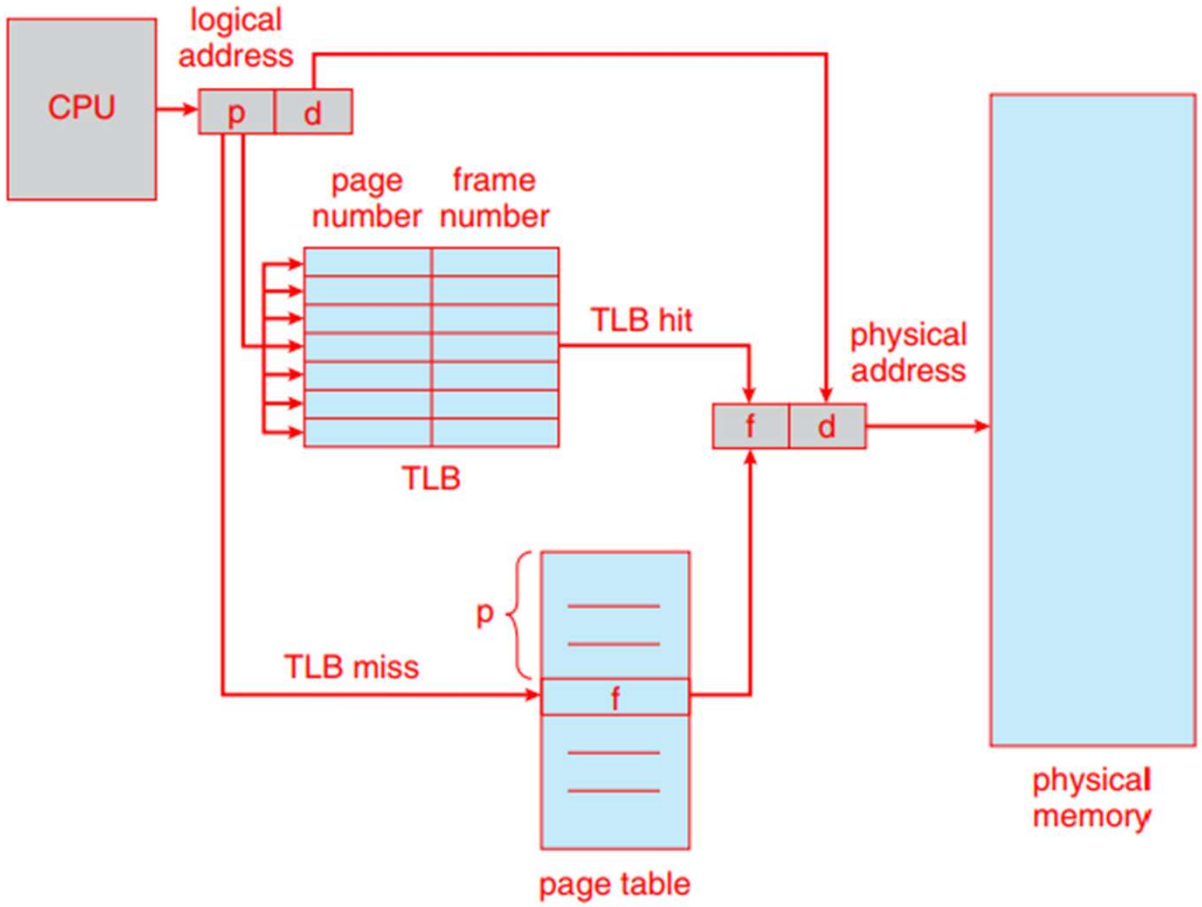
The Challenge: Performance

- The page table is kept in main memory
- The **Page-Table Base Register (PTBR)** points to the page table
- This means every data/instruction access requires **two memory accesses**:
  1. One for the page-table entry
  2. One for the actual data/instruction
- This slows down memory access by a factor of 2, which is an unacceptable performance hit

The Solution: Translation Look-aside Buffer (TLB)

- A small, fast, associative hardware cache
- Stores recent translations of page numbers to frame numbers
- If the page number is in the TLB (**TLB Hit**), the frame number is retrieved quickly
- If the page number is not in the TLB (**TLB Miss**), the page table is consulted, and the page/frame pair is added to the TLB

Paging Hardware with TLB



Ensuring Memory Integrity

Paging hardware provides memory protection through bits in the page table:

- **Protection Bits:**
  - Associated with each frame to control access
  - Commonly define a page as **read-write** or **read-only**
  - An attempt to write to a read-only page causes a hardware trap to the OS
- **Valid-Invalid Bit:**
  - **Valid (v):** The associated page is in the process's logical address space and is a legal page
  - **Invalid (i):** The page is *not* in the process's logical address space.
  - Any access to an 'invalid' page results in a trap to the OS
  - This protects a process from accessing memory outside its own logical address range

Efficiently Sharing Code and Data

- Paging makes it easy for processes to share common code
- **Reentrant Code:** Non-self-modifying code (also called "pure code") that never changes during execution
  - Examples: standard libraries (like the C library), compilers, window systems
- **Mechanism:**
  - The shared code (e.g., a library) is loaded into physical memory in a set of frames
  - The page tables of all processes that *use* that code are mapped to the **same physical frames**
- **Benefit:**
  - Massive savings in physical memory
  - Only one copy of the library needs to be in memory, no matter how many processes are using it
  - Also used to implement shared memory for interprocess communication

Page Table Structures

The Problem: Very Large Page Tables

- Modern computer systems support large logical address spaces (e.g., 2^{32} to 2^{64})
- A simple, single-level page table can become excessively large.
- **Example (32-bit system with 4KB pages):**
  - A 4GB (2^{32}) logical address space requires a page table with 1 million entries (2^{20})
  - If each entry is 4 bytes, the page table itself is **4MB**
- This is too large to keep in memory for every single process. We need more sophisticated methods

A Multi-Level Approach (Page the Page Table)

- **Concept:** Break the logical address space into multiple page tables
- **Two-Level Paging (Common for 32-bit):**
  - The page table itself is "paged"
  - The logical address is divided into three parts:
    - **Page Directory Index (p1):** An index into the outer page table (Page Directory)
    - **Page Table Index (p2):** An index into the inner page table
    - **Page Offset (d):** The offset within the frame
  - This is a "forward-mapped" page table
- **64-bit Systems:** Two-level paging is not sufficient. It would require too many levels of indirection (e.g., 6 levels), making memory access too slow

Handling Address Spaces Larger than 32 Bits

- **Concept:** The virtual page number is hashed into a **hash table**.
- **Structure:**
  1. The hash table contains pointers to the elements.
  2. Each element in the table consists of:
     - The virtual page number
     - The mapped frame number
     - A pointer to the next element in the linked list
- **Operation:**
  1. The virtual page number is hashed
  2. The corresponding linked list is searched for a match
- **Common Variants:** Clustered page tables and hashed inverted page tables

A Different Perspective: Track Physical Frames

- **Concept:** Instead of one entry for each *logical page*, there is only one entry for each *physical frame* in the system
- **Structure:**
  - Each entry in the inverted page table stores the virtual address <process-id, page-number> of the page stored in that physical frame
- **Benefit:**
  - Drastically reduces the amount of memory needed for page tables. The table size is dependent on physical memory, not the logical address space of all processes combined
- **Drawback:**
  - Increases the time needed to search for a page. A simple index lookup is no longer possible
  - Requires a search (or a hash table to assist the search) of the entire table to find a match
  - Makes implementing shared memory more difficult

Summary and Trade-offs

Feature	Hierarchical Paging	Hashed Page Tables	Inverted Page Tables
Memory Usage	Can still be large; depends on address space used.	Good for sparse address spaces; moderate overhead.	**Excellent.** Size depends on physical memory only.
Lookup Time	Slow due to multiple memory accesses (2 to 4).	Generally fast, but depends on hash collisions.	Slow due to search; often requires a hash assist.
Complexity	Moderately complex to manage levels.	Hash function and collision management add complexity.	Most complex due to search requirement.
Best For	32-bit systems and moderately large address spaces.	64-bit systems with large, sparse address spaces.	Systems where memory for page tables is critical.

Swapping

Moving Processes Between Main Memory and Backing Store

- **Concept:** A process can be temporarily swapped from main memory to a **backing store** (a fast, large storage like an SSD) and then brought back into memory for continued execution
- **Purpose:** To make main memory available for other processes, thereby increasing the degree of multiprogramming
- **Classic Swapping:**
  - The original version of swapping moved entire processes between memory and disk
  - This is a very slow operation due to the large amount of data being transferred
  - This "standard swapping" is no longer used in its classic form in modern operating systems

The Modern Approach: Swapping Pages, Not Processes

- Modern systems that use paging don't swap entire processes. Instead, they swap individual **pages**.
- This is often referred to as **paging**, but it is a form of swapping.
- **Operation:**
  - **Page Out:** When the system needs more free frames, it selects pages currently in memory (that are not in active use) and writes their contents to the backing store. The frame is now free.
  - **Page In:** When the process needs a page that has been "paged out," the system brings it back from the backing store into an available frame.
- **Benefit:** This is much more flexible and efficient than moving an entire process. It is the standard mechanism used in OSes like Linux and Windows.

Memory Management Without Traditional Swapping

- **Constraint:** Mobile operating systems like iOS and Android typically do **not** use swapping in the traditional sense
- **Reason:** Flash memory, which is used in mobile devices, has limitations:
  1. **Limited Write Cycles:** Flash memory can only be written to a finite number of times. Frequent swapping would wear out the storage
  2. **Slower Throughput:** The throughput between flash memory and the CPU is lower than on desktop systems, making swapping a poor user experience
- **Alternative Strategies:**
  - **Application-Managed Memory:** iOS asks applications that are running in the background to voluntarily free up memory
  - **Process Termination:** Android may terminate non-essential applications to free memory if it is running low. Any application state must be saved by the app itself for later restoration

Real-World Example

- Linux systems support swapping on a per-page basis
- **Swap Space:** A swap space can be a dedicated disk partition (a swap partition) or a file on the file system (a swap file)
- **Multiple Swap Spaces:**
  - A Linux system can have multiple swap spaces
  - Each swap space can be assigned a **priority**, from -1 to 32,767
  - The kernel will use swap spaces with higher priorities first. If they have the same priority, their usage is interleaved
- **Usage:** The swapping mechanism is managed by the kernel's memory-management subsystem and is used when physical memory is under pressure

Main Memory